

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #8

Functors

Agenda

- **Built in predicates**
 - `functor` and `arg` and `=..`
- **Utilization on complex tasks**
 - Substructure change

Creating/accessing structures

Built-in predicates

- `functor(T, F, N)`
 - means `T` is a structure named `F` with arity `N`
 - 2 functionalities:
 - **if** `nonvar(T)`
then `F` matches the functor of `T` and
`N` matches the arity of `T`
 - **if** `var(T)` **and** `nonvar(F)` **and** `nonvar(N)`
then `T` becomes a structure
with name `F` and `N` arguments

functor(T, F, N)

if nonvar(T) **then** F matches the functor of T
N matches the arity of T

```
?- functor(a+b,F,N).  
true, F=+, N=2.
```

```
?- functor(a,F,N).  
true, F=a, N=0.
```

```
?- functor([a,b,c],F,N).  
true, F=., N=2.    % Why 2? Which?
```

```
?- functor(f(a,b),F,N).  
true, F=f, N=2.
```

```
?- functor([a,b,c],F,3).  
false.            % Why?
```

functor(T, F, N)

if var(T) **and** nonvar(F) **and** nonvar(N)
then T becomes a structure with name F and N arguments

```
?- functor(F, f, 3) .  
true, F=f(_A, _B, _C) .
```

```
?- functor(F, +, 2) .  
true, F=_A+_B
```

Creating/accessing structures

Built-in predicates

- `arg(N, T, A)`
 - means the N^{th} argument of `T` is `A`
 - `N` and (partially) `T` must be instantiated

```
?- arg(2, f(a,b), X) .  
true, X=b.
```

```
?- arg(2, a+b+c, Z) .  
true, Z=b+c
```

```
?- arg(1, a+b+c, X) .  
true, X=a
```

```
?- arg(1, [a,b,c], X) .  
true, X=a
```

```
?- arg(3, a+b+c, Y) .  
false. % Why?
```

```
?- arg(2, [a,b,c], Y) .  
true, Y=[b,c]
```

Creating/accessing structures

Built-in predicates

$X = . . L$ % INFIX predicate (a.k.a. UNIV)

- means list L contains as first element the functor of X , followed by arguments of X
- 2 functionalities:
 - **if** $\text{var}(X)$ **then**
 L is used to construct the appropriate structure of X ;
the head of L must be an atom, and becomes the functor of X
 - **if** $\text{nonvar}(X)$ **then**
functor of X matches the first element (head) of L ,
while the arguments of X matches with the tail of
the list (left to right arguments of X , left to right in
tail of L)

$X = \dots L$

- **if** $\text{var}(X)$ **then** L is used to construct the appropriate structure of X ; the head of L must be an atom, and becomes the **functor** of X

```
?- X=..[a,b,c,d].  
true, X=a(b,c,d)
```

```
?- X=..[member,a,[b,c]].  
true, X=member(a,[b,c]).
```


$X = \dots L$

- **if** `nonvar(X)` **then** functor of `X` matches the first element (head) of `L`, while the arguments of `X` matches with the tail of the list (left to right arguments of `X`, left to right in tail of `L`)

```
?- f(a,b,c)=..X.                % Can we ask with X on the left?
true, X=[f,a,b,c].
```

```
?- append([H|T],L,[H|R])=..X.
true, X=[append,[H|T],L,[H|R]].
```

```
?- (a+b)=..L.
true, L=[+,a,b].
```

```
?- (a+b+c)=..L.
true, L=[+,a,b+c]
```

```
?- [a,b,c,d]=..[H|T].          % Can we ask with [H|T] on the left?
true, H=., T=[a,[b,c,d]]
```

univ (= ..) implementation

```
univ(Eq, [F|Terms]) :-
    length(Terms, N),
    functor(Eq, F, N),           % Extract f and its arity
                                  % functor(f(a,b,c), f, 3)
    get_args(Eq, 0, N, Terms).
```

% extract each argument one by one based on a counter up to N (arity)

```
get_args(_, N, N, []) :- !.
get_args(Eq, C, N, [X|R]) :-
    C1 is C+1,
    arg(C1, Eq, X),              ?- f(a,b,c)=..L1, univ(f(a,b,c), L2).
                                  L1 = L2, L2 = [f, a, b, c]
    get_args(Eq, C1, N, R).      ?- F1=..[f,a,b,c], univ(F2, [f,a,b,c])
                                  F1 = F2 = f(a,b,c).
```

The predicates

```
?- functor(f(a,b,c), F, N).      ?- functor(F, f, 3).  
true, F=f, N=3.                  true, F=f(_1,_2,_3).
```

```
?- arg(2, f(a,b,c), X).  
true, X=b.
```

```
?- X=..[f,a,b,c].                ?- f(a,b,c)=..X.  
true, X=f(a,b,c).                true, X=[f,a,b,c].
```

Substituting expressions

- Given a (complex) expression, it is requested to replace a given subexpression with another one.

```
% subst /4 (+OldSubExpr, +NewSubExpr, +OldExpr, -NewExpr) .
```

Example: in a large arithmetic expression replace $(7-2*x)$ wherever it occurs with another one, say $(2*y+3/z)$

- 2 solutions
 - `subst1/4: =..`
 - `subst2/4: functor + arg`

```
% subst /4 (+OldSubExpr, +NewSubExpr, +OldExpr, -NewExpr) .
```

```
% subst_args /4
```

```
% (+OldSubExpr, +NewSubExpr, +ListOldArgs, -ListNewArgs) .
```

Substituting expressions

=..

```
subst1(Old, New, Old, New) :- !.           % in case the whole expression is represented
                                           % by only the subexpression to be replaced,
                                           % the old one is replaced by the new one

subst1(_, _, Old, Old) :-                  % in case the expression is an atomic Exprue
    atomic(Old), !.                        % it remains unchanged

subst1(Old, New, Expr, NewExpr) :-
    Expr=..[F|Args],                      % extract from the old expression its functor
                                           % and the list of arguments
    subst_args1(Old, New, Args, NewArgs), % take the old list of arguments and obtain the substitution
                                           % create the new expression with the same functor and the
    NewExpr=..[F|NewArgs].                 % new list of arguments, updated by substitutions

subst_args1(_, _, [], []).
subst_args1(Old, New, [Arg|Args], [NewArg|NewArgs]) :-
    subst1(Old, New, Arg, NewArg),         % as Arg is an expression, go back
                                           % to the other predicate
    subst_args1(Old, New, Args, NewArgs). % continue with the rest of
                                           % arguments, tail of list
```

Substituting expressions =..

Just code, comments removed

```
subst1(Old,New,Old,New):-!.
subst1(_,_,Old,Old):-
    atomic(Old),!.
subst1(Old,New,Expr,NewExpr):-
    Expr=..[F|Args],
    subst_args1(Old,New,Args,NewArgs),
    NewExpr=..[F|NewArgs].

subst_args1(_,_,[],[]).
subst_args1(Old,New,[Arg|Args],[NewArg|NewArgs]):-
    subst1(Old,New,Arg,NewArg),
    subst_args1(Old,New,Args,NewArgs).
```

Qs:

1. Order of processing the structure?
2. Where the execution ends?

[q] ?- subst1(a, x, f(a, b, g(a)), R).
R = f(x,b,g(x))

Substituting expressions functor+arg

```

subst2 (Old, New, Old, New) :-!.           % same as before
subst2 (_, _, Old, Old) :-                  % same as before
    atomic (Old), !.
subst2 (Old, New, Expr, NewExpr) :-
    functor (Expr, F, N),                  % extract from the old expression Expr its functor F
                                           % and the number of arguments N
    functor (NewExpr, F, N),               % create a new expression NewExpr from the
                                           % known functor F and N arguments, free variables
at the time subst_args2 (N, Old, New, Expr, NewExpr) .

% for each of the arguments, 1, 2, ..., N, process it one at a time
subst_args2 (0, _, _, _, _) :-!.           % arg 0 needs no change
subst_args2 (N, Old, New, Expr, NewExpr) :-
    arg (N, Expr, OldArg),                 % extract the Nth arg of the old expression
    arg (N, NewExpr, NewArg),              % set the Nth arg of the new expression;
                                           % a var at the time
    subst2 (Old, New, OldArg, NewArg),     % as OldArg is an expression, go back to the other
                                           % predicate and make the same processing
    N1 is N-1,                             % continue by processing the previous argument
    subst_args2 (N1, Old, New, Expr, NewExpr) .           % continue with the rest of
                                                                % arguments, tail of list

```

Substituting expressions **functor+arg**

Just code, comments removed

```
subst2(Old, New, Old, New) :- !.  
subst2(_, _, Old, Old) :-  
    atomic(Old), !.  
subst2(Old, New, Expr, NewExpr) :-  
    functor(Expr, F, N),  
    functor(NewExpr, F, N),  
    subst_args2(N, Old, New, Expr, NewExpr).  
  
subst_args2(0, _, _, _, _) :- !.  
subst_args2(N, Old, New, Expr, NewExpr) :-  
    arg(N, Expr, OldArg),  
    arg(N, NewExpr, NewArg),  
    subst2(Old, New, OldArg, NewArg),  
    N1 is N-1,  
    subst_args2(N1, Old, New, Expr, NewExpr).
```

Qs:

1. Order of processing the structure?
2. Where the execution ends?

[q] ?- subst2(a, x, f(a, b, g(a)), R).
R = f(x,b,g(x))